

Simulación de Entrevista Técnica — Testing

Guía de estudio en formato entrevista · JUnit 5, Mockito, Reactor Test

(StepVerifier), Testcontainers, JaCoCo y Gatling en el sistema POS

Guía de estudio en formato entrevista sobre la **estrategia de testing** del

sistema POS de microservicios: **tests unitarios** con JUnit 5 + Mockito + StepVerifier, **tests de integración** con Testcontainers, **cobertura** con JaCoCo, **calidad** con SonarQube y **pruebas de carga** con Gatling. Cada sección incluye la **pregunta del entrevistador**, una **respuesta modelo** anclada al repo y **follow-ups**.

Formato sugerido: 50–60 min. Bloques: pirámide y fundamentos (10'), unit tests

reactivos (15'), Testcontainers (15'), cobertura/CI (10'), Gatling y escenarios (10').

0. Warm-up — Testing en el proyecto

P ¿Cómo está organizada la estrategia de testing en este sistema POS?

R

Hay **tres capas** verificadas en el repo:

1. **Tests unitarios (JUnit 5 + Mockito + Reactor Test)**: son la gran mayoría — **22 clases de test** con unos **219 métodos @Test** en `stock-service`, `venta-service` y `despacho-service`. Se etiquetan con `@Tag("unit")` y aíslan la lógica con mocks (p. ej. `SagaOrchestratorTest`, `VentaControllerTest`, `SagaReconcilerTest`).
2. **Tests de integración (Testcontainers)**: `StockServiceIntegrationTest` con `@SpringBootTest` + `@Testcontainers`, etiquetado `@Tag("integration")`, que levanta **MongoDB, Kafka y Redis** reales en contenedores.
3. **Pruebas de carga (Gatling)**: módulo `stress-test/` con 6 simulaciones (`SagaEndToEndSimulation`, `VentaServiceSimulation`, etc.) contra el gateway en `:8080`.

Todo se orquesta desde Gradle: `./gradlew test` (unit), `./gradlew integrationTest` (Testcontainers), `./gradlew :stress-test:gatlingRun` (carga), con **JaCoCo** para cobertura y **SonarQube** para calidad.

FOLLOW-UP ¿Los tres frontends Angular tienen tests? — Hoy **no**: no hay archivos `*.spec.ts` en `pos-frontend`, `ventas-mantenedor` ni `users-mantenedor`. Es una brecha real: el foco de pruebas está en el backend.

1. Pirámide de tests y fundamentos

P Explica la pirámide de tests y dónde se sitúa este proyecto.

R La **pirámide** propone muchos **tests unitarios** (rápidos, baratos, aislados) en la base, menos **tests de integración** en el medio y muy pocos **end-to-end** en la cima (lentos y frágiles). El POS la respeta: ~219 tests unitarios con Mockito frente a **una** clase de integración con Testcontainers y un puñado de simulaciones Gatling E2E. Los unit tests corren en cada `build`; los de integración y carga se ejecutan aparte por su coste.

P ¿Qué framework y qué librerías de test usa el backend? ¿Dónde se declaran?

R

En el `build.gradle` raíz, el bloque `subprojects` declara para todos los servicios reactivos:

- `spring-boot-starter-test` (trae **JUnit 5**, **Mockito**, **AssertJ**).
- `spring-kafka-test` (utilidades de test para Kafka).
- `io.projectreactor:reactor-test` (**StepVerifier** para `Mono / Flux`).
- `org.testcontainers:testcontainers/junit-jupiter/mongodb/kafka/postgresql` versión **1.19.8**.
- Lombok también en el classpath de test (`testCompileOnly` + `testAnnotationProcessor`).

La tarea `test` usa `useJUnitPlatform()`, es decir **JUnit 5 (Jupiter)**.

FOLLOW-UP ¿Por qué AssertJ y no solo asserts de JUnit? — AssertJ da un API fluido y legible (`assertThat(created.getReservedQuantity()).isEqualTo(0)`) con mejores mensajes de fallo. Se ve por todo el repo.

2. Tests unitarios con Mockito

P ¿Cómo se estructura un test unitario típico aquí? Pon un ejemplo real.

R

El patrón es **JUnit 5 + Mockito** sin arrancar Spring. Ejemplo: `SagaOrchestratorTest` .

```
@Tag("unit")
@ExtendWith(MockitoExtension.class)
class SagaOrchestratorTest {
    @Mock private OrderRepository orderRepository;
    @Mock private StockEventPublisher stockEventPublisher;
    @Mock private DespachoEventPublisher despachoEventPublisher;
    @InjectMocks private SagaOrchestrator sagaOrchestrator;
    // ...
}
```

`@ExtendWith(MockitoExtension.class)` habilita Mockito; `@Mock` crea dobles de las dependencias y `@InjectMocks` los inyecta en la clase bajo prueba. Se programa el comportamiento con `when(...).thenReturn(...)` y se verifica con `verify(...)` . Así el test es **rápido y determinista**, sin base ni Kafka.

P ¿Cómo se organizan los casos dentro de una clase de test?

R Con `@Nested` y `@DisplayName` para agrupar escenarios y darles nombres legibles. En `SagaOrchestratorTest` hay grupos como `@Nested @DisplayName("Handle Stock Response Tests")` , `"Handle Despacho Response Tests"` , `"Concurrent Saga Events Tests"` y `"CircuitBreaker Fallback Tests"` . Cada `@Test` lleva su `@DisplayName` descriptivo (`"Should set STOCK_RESERVED and send despacho request on success"`), lo que produce informes muy legibles.

P ¿Cómo verificas que un evento **no** se emitió? Da un caso del repo.

R

Con `verify(mock, never())`. En el caso de stock insuficiente de `SagaOrchestratorTest`, tras procesar un `StockReserveResponseEvent` con `success(false)` se comprueba que la orden pasa a **STOCK_FAILED** y que **no** se pide despacho:

```
verify(despachoEventPublisher, never()).requestDespacho(any());
```

Esto valida la regla de negocio: sin stock reservado no se solicita despacho.

FOLLOW-UP ¿Y para capturar el argumento con que se llamó un mock? — Con `ArgumentCaptor`. En `SagaReconcilerTest` se captura el `StockReserveEvent` re-emitido y se asegura `captor.getValue().getOrderId().isEqualTo("order-1")`.

3. Tests de código reactivo (StepVerifier)

P El backend es reactivo (`Mono` / `Flux`). ¿Cómo se testea eso?

R

Con **StepVerifier** de `reactor-test`. No se puede hacer un `assertEquals` directo sobre un `Mono` porque es perezoso; hay que **suscribirse y verificar** la señal. Patrón real de `SagaOrchestratorTest`:

```
StepVerifier.create(sagaOrchestrator.handleStockResponse(event))
    .verifyComplete();
```

Para un flujo con valor, se usa `.assertNext(...)` como en el test de integración de stock:

```
StepVerifier.create(stockApplicationService.createProduct(newProduct))
    .assertNext(created -> {
        assertThat(created.getId()).isNotNull();
        assertThat(created.getReservedQuantity()).isEqualTo(0);
    })
    .verifyComplete();
```

P ¿Cómo se verifica que un Mono **falla** con cierto error?

R

Con `.expectErrorMatches(...)`. Cuando la orden no existe, `handleStockResponse` propaga un error y el test lo comprueba:

```
StepVerifier.create(sagaOrchestrator.handleStockResponse(event))
    .expectErrorMatches(e -> e instanceof RuntimeException &&
        e.getMessage().contains("Order not found: nonexistent"))
    .verify();
```

Distinto de `verifyComplete()` (terminación normal), aquí se exige la señal `onError`. En los tests de fallback del circuit breaker se distingue incluso entre un error **transitorio** que se **propaga** (para retry/DLQ) y una `OrderNotFoundException` **terminal** que **completa vacío**.

FOLLOW-UP ¿Por qué no usar `.block()` y comparar? — `block()` funciona en casos simples (el test de integración lo usa una vez con `createProduct(product).block()`), pero `StepVerifier` verifica el **contrato reactivo** completo: número de elementos, orden, señal terminal y errores.

4. Tests de controladores y de Kafka

P ¿Cómo se prueban los controladores REST reactivos aquí?

R

En `VentaControllerTest` se prueba el controlador como un **POJO** con Mockito, sin `@WebFluxTest`: se mockean `OrderCommandService` y `OrderQueryService`, se invoca el método y se verifica el `ResponseEntity` con `StepVerifier`:

```
when(orderCommandService.crearVenta(any(Order.class))).thenReturn(Mono.just(testOrder));

StepVerifier.create(ventaController.crearVenta(testOrder))
    .assertNext(response -> {
        assertThat(response.getStatusCode()).isEqualTo(HttpStatus.CREATED);
        assertThat(response.getBody().getId()).isEqualTo("order-1");
    })
    .verifyComplete();
```

Es un unit test puro: valida el mapeo a `HttpStatus.CREATED` y la propagación de errores sin levantar servidor HTTP.

P ¿Y los consumidores/productores de Kafka?

R También como unit tests con Mockito. En `VentaConsumerTest` se mockean `SagaOrchestrator`, `CartRepository` y `ObjectMapper`; se simula el mensaje entrante (un `Map`) y se verifica que se convierte al evento y se delega en el orquestador. Un caso clave:

propagar la excepción de conversión para que Kafka pueda reintentar/DLQ

(`@DisplayName("Should PROPAGATE conversion exception so Kafka can retry/DLQ")`). Hay clases análogas `VentaProducerTest`, `StockConsumerTest`, `DespachoProducerTest`, etc.

FOLLOW-UP Entonces `spring-kafka-test` casi no se usa. ¿Por qué? — Está en el classpath para poder usar `@EmbeddedKafka` si hiciera falta, pero la estrategia elegida es **mockear** el orquestador y probar el broker de verdad solo en el test de integración con `Testcontainers`. Menos superficie lenta.

5. Tests de integración con Testcontainers

P ¿Qué son los tests de integración con Testcontainers y cómo se usan aquí?

R

Testcontainers levanta **infraestructura real en contenedores Docker** durante el test, en vez de mocks o embebidos. La única clase es `StockServiceIntegrationTest` :

```
@Tag("integration")
@SpringBootTest
@Testcontainers
class StockServiceIntegrationTest {
    @Container static MongoDBContainer mongoDBContainer =
        new MongoDBContainer(DockerImageName.parse("mongo:7.0"));
    @Container static KafkaContainer kafkaContainer =
        new KafkaContainer(DockerImageName.parse("confluentinc/cp-kafka:7.6.1"));
    @Container static GenericContainer<?> redisContainer =
        new GenericContainer<>(DockerImageName.parse("redis:7.2-alpine")).with
```

Levanta **MongoDB 7.0**, **Kafka (cp-kafka 7.6.1)** y **Redis 7.2** reales, arranca el contexto Spring completo (`@SpringBootTest`) y prueba de punta a punta el `StockApplicationService` (crear producto, cache hit en segunda lectura, etc.).

P ¿Cómo conecta la app a esos contenedores si sus puertos son aleatorios?

R

Con `@DynamicPropertySource` : `Testcontainers` asigna puertos efímeros y el método inyecta las URLs reales en el `Environment` de Spring **antes** de arrancar el contexto:

```
@DynamicPropertySource
static void configureProperties(DynamicPropertyRegistry registry) {
    registry.add("spring.data.mongodb.uri", mongoDBContainer::getReplicaSetUri);
    registry.add("spring.kafka.bootstrap-servers", kafkaContainer::getBootstrapServers);
    registry.add("spring.data.redis.host", redisContainer::getHost);
    registry.add("spring.data.redis.port", () -> redisContainer.getMappedPort(6379));
    registry.add("eureka.client.enabled", () -> "false");
}
```

Nota el `eureka.client.enabled=false` : se **desactiva el service discovery** en test para no depender de Eureka.

P ¿Por qué el test normal **exclude** los tests de integración?

R

Por el coste y la fiabilidad de Docker en CI. En el `build.gradle` raíz:

```
tasks.named('test') {
    useJUnitPlatform {
        if (!project.hasProperty('includeIntegration')) {
            excludeTags 'integration'
        }
    }
}
tasks.register('integrationTest', Test) {
    useJUnitPlatform { includeTags 'integration' }
}
```

El comentario del propio build lo dice: los tests de integración necesitan un **daemon Docker fiable**, que no está garantizado en Jenkins/Docker Desktop, así que se **excluyen del build** de CI y se corren localmente con `./gradlew integrationTest` (o `./gradlew test -PincludeIntegration`).

FOLLOW-UP ¿Contenedores estáticos o por método? — Aquí son `static`, así que se comparten entre todos los `@Test` de la clase (una sola arrancada de MongoDB/Kafka/Redis), que es más rápido que recrearlos por método.

6. Cobertura (JaCoCo) y calidad (SonarQube)

P ¿Cómo se mide la cobertura y qué umbral se exige?

R

Con **JaCoCo** (`toolVersion "0.8.12"`), aplicado a todos los subproyectos. La tarea `test` dispara el reporte por `finalizedBy jacocoTestReport`, y hay una **verificación de umbral**:

```
jacocoTestCoverageVerification {
    violationRules { rule { limit { minimum = 0.70 } } }
}
tasks.named('check') { dependsOn jacocoTestCoverageVerification }
```

Es decir, el mínimo **real** en el build es **0.70 (70%)**, atado a la tarea `check`. El reporte HTML queda en `<service>/build/reports/jacoco/test/html/index.html`.

FOLLOW-UP El README dice "80%". ¿Cuál manda? — El **build** manda: 70%. El README documenta una meta de 80% que **no** coincide con la regla configurada; lo señalaría como una inconsistencia a alinear.

P ¿Cómo integra el proyecto la cobertura con SonarQube?

R El plugin `org.sonarqube (7.4.0.8496)` está en el `build.gradle` raíz con `sonar.projectKey = "saga-microservices"` y `sonar.java.coveragePlugin = "jacoco"`, así que Sonar consume el XML de JaCoCo. Se ejecuta con `./gradlew sonar -Dsonar.host.url=http://localhost:9000` y el token se pasa por `SONAR_TOKEN` (variable de entorno, **nunca** hardcodeado).

P ¿Dónde encaja el testing en el pipeline de CI?

R

En el `Jenkinsfile` raíz, el stage **Build & Test** ejecuta:

```
./gradlew build --build-cache -x jacocoTestCoverageVerification \
-x :stress-test:test -x :stress-test:build
```

Corre los **unit tests** (parte de `build`), publica resultados con `junit testResults: '**/build/test-results/test/*.xml'`, y **excluye** la verificación de umbral y el módulo Gatling para que el pipeline sea rápido y no dependa de un entorno de carga.

7. Pruebas de carga con Gatling

P ¿Para qué usa el proyecto Gatling y dónde vive?

R Para **pruebas de carga/estrés** de las APIs a través del gateway. Vive en el módulo `stress-test/` con el plugin `io.gatling.gradle (3.11.5.2)` y Java 21. Hay 6 simulaciones: `SagaEndToEndSimulation`, `VentaServiceSimulation`, `StockServiceSimulation`, `DespachoServiceSimulation`, `AuthServiceSimulation` y `ResilienceSimulation`. Se lanza con `./gradlew :stress-test:gatlingRun`.

P Describe un escenario E2E de Gatling anclado al repo.

R

`SagaEndToEndSimulation` recorre la SAGA completa contra `gatewayUrl` (:8080 por defecto):

1. **Login:** `POST /api/v1/auth/login`, guarda `$.token`.
2. **Crear producto:** `POST /api/v1/stock`, guarda `$.id` → `productId`.
3. **Crear orden (dispara SAGA):** `POST /api/v1/ventas`, guarda `orderId`.
4. **Poll de estado:** `GET /api/v1/ventas/#{orderId}` tras una pausa.
5. **Verificar despacho:** `GET /api/v1/despachos/order/#{orderId}`.

Incluye un escenario de **rate limit** (`repeat(50)` esperando `200` o **429**) y otro de **stock insuficiente** que pide 9999 unidades y verifica `jsonPath("$.status").is("STOCK_FAILED")`.

P ¿Cómo modela la carga y qué SLAs valida?

R

Con inyección abierta (`injectOpen`): `rampUsers(20).during(15s)` seguido de `constantUsersPerSec(5).during(30s)`, más otros escenarios concurrentes. Al final define **assertions** globales que hacen fallar la simulación si no se cumplen:

```
global().responseTime().max().lt(15000),
global().responseTime().percentile(95.0).lt(5000),
global().successfulRequests().percent().gt(90.0)
```

O sea: tiempo máximo < 15 s, **p95 < 5 s** y **> 90% de peticiones exitosas**.

FOLLOW-UP ¿Por qué Gatling no está en el pipeline principal? — El `Jenkinsfile` raíz **excluye** `:stress-test:test` y `:stress-test:build`; las pruebas de carga necesitan el stack corriendo y son de otra naturaleza (rendimiento, no corrección), así que se ejecutan aparte/bajo demanda.

8. Diseño abierto / escenarios

P Tienes que testear una regla de reconciliación de SAGAs colgadas. ¿Cómo?

R Como en `SagaReconcilerTest` : es un unit test con Mockito que construye el `SagaReconciler` con un `Duration.ofMinutes(2)` de umbral. Se mockea `orderRepository.findByStatusInAndUpdatedAtBefore(...)` devolviendo una orden **PENDING** vieja, se corre `reconcile()` con `StepVerifier(.verifyComplete())` y con un `ArgumentCaptor<StockReserveEvent>` se verifica que se **re-emitió** el evento correcto. Otro test asegura que solo se consultan los estados intermedios `PENDING` y `STOCK_RESERVED` (`containsExactlyInAnyOrder`). Determinista, sin Kafka ni Mongo.

P El equipo quiere subir la cobertura de 70% a 85%. ¿Qué harías?

R Primero, **alinear** la contradicción build (70%) vs README (80%). Luego subir `jacocoTestCoverageVerification { minimum }` gradualmente y atacar los huecos que revele el reporte de JaCoCo: la brecha más obvia es **auth-service**, que hoy no tiene **src/test** pese a manejar JWT y seguridad — ahí añadiría tests de `JwtService`, `AuthApplicationService` y el `SecurityConfig`. Evitaría subir el número con tests triviales sin asserts.

P ¿Qué le falta a esta estrategia de testing y cómo lo priorizarías?

R Tres brechas reales, por prioridad: (1) **tests de auth-service** (0 tests hoy en un servicio de seguridad); (2) **tests de frontend** — los tres proyectos Angular no tienen `*.spec.ts` (la diapositiva menciona Jest/Jasmine pero no hay nada); (3) **más tests de integración con Testcontainers** — solo `stock-service` los tiene, faltarían en `venta` y `despacho` para cubrir el flujo SAGA con Kafka real. Priorizaría auth por riesgo de seguridad.

FOLLOW-UP ¿Contract testing entre servicios? — Buen siguiente paso: con Kafka de por medio, un enfoque de **contratos** (p. ej. sobre los esquemas de eventos `StockReserveEvent` / `DespachoResponseEvent`) evitaría romper consumidores al cambiar productores, algo que hoy solo cubre el E2E de Gatling.

Apéndice — Chuleta rápida

Concepto	En este repo
Framework	JUnit 5 (Jupiter) vía <code>useJUnitPlatform()</code>
Mocking	Mockito (<code>@ExtendWith(MockitoExtension.class)</code> , <code>@Mock</code> , <code>@InjectMocks</code>)
Aserciones	AssertJ (<code>assertThat(...)</code>)
Reactivo	StepVerifier (<code>reactor-test</code>): <code>assertNext</code> , <code>verifyComplete</code> , <code>expectErrorMatches</code>
Captura de args	<code>ArgumentCaptor</code> (<code>SagaReconcilerTest</code>)
Organización	<code>@Nested</code> + <code>@DisplayName</code>
Etiquetas	<code>@Tag("unit")</code> / <code>@Tag("integration")</code>
Integración	<code>@SpringBootTest</code> + <code>@Testcontainers</code> (<code>StockServiceIntegrationTest</code>)
Contenedores	<code>mongo:7.0</code> , <code>confluentinc/cp-kafka:7.6.1</code> , <code>redis:7.2-alpine</code>
Config dinámica	<code>@DynamicPropertySource</code> (uri Mongo, bootstrap Kafka, Redis, <code>eureka.client.enabled=false</code>)
Escala del backend	22 clases , ~ 219 <code>@Test</code>
Cobertura	JaCoCo 0.8.12 , umbral 0.70 vía <code>jacocoTestCoverageVerification</code>
Calidad	SonarQube <code>projectKey=saga-microservices</code> , <code>coveragePlugin=jacoco</code>
Carga	Gatling en <code>stress-test/</code> (6 simulaciones), assertions p95<5s, éxito>90%
Comandos	<code>./gradlew test</code> · <code>./gradlew integrationTest</code> · <code>./gradlew jacocoTestReport</code> · <code>./gradlew sonar</code> · <code>./gradlew :stress-test:gatlingRun</code>
CI (Jenkins)	<code>./gradlew build -x jacocoTestCoverageVerification -x :stress-test:test ; publica **/build/test-results/test/*.xml</code>

Concepto	En este repo
Brechas	<code>auth-service</code> sin tests · frontends Angular sin <code>*.spec.ts</code> · integración solo en <code>stock</code>